

SearchFlow – Distributed Search Typeahead System

1. Project Overview

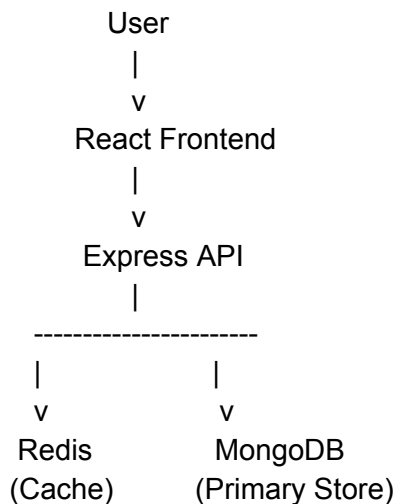
SearchFlow is a distributed search typeahead system designed to provide low-latency search suggestions similar to modern search engines and e-commerce platforms.

The system supports:

- Prefix-based search suggestions
- Search submission and query-count updates
- Trending searches
- Distributed caching using Redis
- Consistent hashing for cache distribution
- Batch writes for reducing database write pressure
- Large-scale dataset support (500,000 records)

The goal of this project is to demonstrate scalable backend system design concepts while maintaining a responsive user experience.

2. System Architecture



|
v

Consistent Hashing

|
v

Batch Writer

Components

Frontend

- React
- Vite
- CSS

Responsible for:

- Search UI
- Suggestions dropdown
- Trending searches display
- Search submission

Backend

- Node.js
- Express.js

Responsible for:

- API handling
- Cache management
- Search ranking
- Batch processing

Database

- MongoDB Atlas

Stores:

- Query text
- Historical search count

- Recent search count

Cache Layer

- Redis Cloud

Caches suggestion results for frequently searched prefixes.

3. Dataset

Dataset Size:

500,000 search queries

Dataset Structure:

```
{  
  "query": "iphone charger",  
  "count": 500,  
  "recentCount": 0  
}
```

The dataset exceeds the assignment requirement of 100,000 records.

The dataset was loaded into MongoDB using a custom seeding script.

4. API Documentation

GET /suggest

Returns search suggestions matching a prefix.

Example:

GET /suggest?q=iph

Response:

```
[
  {
    "query": "iphone",
    "count": 10000
  }
]
```

Behavior:

- Returns up to 10 suggestions
 - Prefix matching only
 - Sorted by ranking score
-

POST /search

Records a search submission.

Example:

```
{
  "query": "iphone charger"
}
```

Response:

```
{
  "message": "Searched"
}
```

Behavior:

- Search count updated asynchronously
 - Query inserted if it does not exist
-

GET /trending

Returns trending search queries.

Behavior:

- Uses historical popularity
 - Uses recent activity
-

GET /cache/debug

Used for demonstrating consistent hashing.

Example:

GET /cache/debug?prefix=iphone

Response:

```
{  
  "prefix": "iphone",  
  "cacheNode": "Node A"  
}
```

5. Design Decisions

Why MongoDB?

MongoDB provides:

- Flexible schema
- Fast development
- Easy indexing
- Horizontal scalability

Indexes were created to improve lookup performance.

Why Redis?

Redis provides:

- Extremely fast reads
- Reduced database load
- Better user experience

Suggestion results are cached using:

suggest:<prefix>

Example:

suggest:iph

Why Consistent Hashing?

Consistent hashing distributes prefixes across logical cache nodes.

Example:

iphone -> Node A

java -> Node B

chatgpt -> Node C

Benefits:

- Better load distribution
 - Easy scalability
 - Minimal cache movement when nodes change
-

6. Trending Searches

The assignment required support for trending searches.

The implementation uses two metrics:

Historical Popularity

count

Represents all-time search frequency.

Recent Popularity

recentCount

Represents recent search activity.

Trending searches are ranked using both values so that recently searched queries can move higher in rankings while still respecting long-term popularity.

Trade-Off

Advantages:

- Better freshness
- More relevant results

Disadvantages:

- Additional storage
 - More ranking computation
-

7. Batch Writes

Updating MongoDB on every search request is expensive.

To reduce write pressure:

Search Requests



Buffer



Aggregation



Batch Write



MongoDB

Repeated searches are aggregated before writing.

Benefits

- Fewer database writes
- Better throughput
- Reduced load on MongoDB

Failure Trade-Off

If the application crashes before a batch is flushed, some buffered searches may be lost.

This trade-off was accepted to improve overall system performance.

8. Performance Report

Dataset Size:

- 500,000 records

Caching:

- Redis Cloud

Database:

- MongoDB Atlas

Deployment:

- Render (Backend)
- Vercel (Frontend)

Observed Benefits:

Cache Hits

Repeated searches are served directly from Redis.

Benefits:

- Reduced latency
- Reduced MongoDB reads

Batch Writes

Multiple searches are aggregated into a single write operation.

Benefits:

- Reduced write count
- Improved scalability

Consistent Hashing

Ensures predictable cache ownership and balanced cache usage.

9. Deployment

Frontend:

<https://hld-assignment.vercel.app/>

Backend:

<https://hld-assignment.onrender.com/>

10. Conclusion

SearchFlow successfully implements a scalable search typeahead system with:

- Prefix-based suggestions
- Trending searches
- Redis caching
- Consistent hashing
- Batch writes
- MongoDB persistence
- Large-scale dataset support

The project demonstrates several important distributed systems concepts while maintaining low-latency query performance and efficient resource utilization.